



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Trabalho Prático — Fase 4 Grupo 31

Afonso Martins

a106931

Francisco Lage

a106813

Gonçalo Castro

a107337

Ricardo Neves

a106850

Maio, 2026

CG

1. Resumo

Este relatório descreve a quarta e última fase do trabalho prático da unidade curricular de Computação Gráfica, na qual o motor de renderização 3D foi expandido com suporte a iluminação, materiais e texturas.

O gerador de primitivas foi estendido para produzir, em cada vértice, um vetor normal calculado analiticamente e coordenadas de textura UV. O motor passou a ler estes dados e a organizá-los em quatro VBOs independentes — posições, normais, coordenadas UV e índices — eliminando vértices duplicados através de uma tabela de hash e utilizando `glDrawElements` para a renderização indexada.

A iluminação foi implementada seguindo o modelo de Phong do *pipeline* de função fixa do OpenGL, com suporte a três tipos de fontes de luz configuráveis em XML: direcional (fonte no infinito, sem atenuação), pontual (atenuação quadrática com a distância) e *spotlight* (cone de iluminação definido por ângulo de *cutoff*). Os materiais, com as componentes difusa, ambiente, especular, emissiva e brilho, são igualmente configuráveis por modelo em XML. As texturas são carregadas com a biblioteca *stb_image* com geração automática de *mipmaps* e partilhadas entre instâncias do mesmo ficheiro.

Como cena de demonstração, o sistema solar foi enriquecido com texturas fotorrealistas dos planetas e iluminação pontual a partir do Sol. Foram ainda implementadas otimizações de desempenho: cache partilhada de VBOs e texturas, cache de estado GPU para minimizar chamadas redundantes ao driver, e *frustum culling* hierárquico com *plane masking* para descartar grupos fora do campo de visão sem emitir *draw calls* desnecessários.

Palavras-chave: OpenGL, iluminação de Phong, normais, coordenadas UV, texturas, VBOs, *frustum culling*, sistema solar.

Índice

2. Introdução	3
3. Normais e Coordenadas de Textura	3
3.1. Formato do ficheiro <code>.3d</code>	3
3.2. Geração de normais por primitiva	3
3.3. Coordenadas UV	4
3.4. Indexação de vértices	5
4. Iluminação	5
4.1. Modelo de iluminação de Phong	5
4.2. Definição em XML	6
4.3. Tipos de luz	6
4.3.1. Luz direcional	6
4.3.2. Luz pontual	6
4.3.3. <i>Spotlight</i>	7
4.4. Propriedades de Reflexão	7
5. Texturas	9
5.1. Carregamento	9
5.2. Cache de texturas	10
5.3. Definição em XML	10
5.4. Aplicação na renderização	10
6. Testes	11
6.1. <code>test_4_1.xml</code> — Luz direcional, cor difusa	11
6.2. <code>test_4_2.xml</code> — Componente emissiva	12
6.3. <code>test_4_3.xml</code> — Dois <i>point lights</i> , especular	13
6.4. <code>test_4_4.xml</code> — <i>Point light</i> central	14
6.5. <code>test_4_5.xml</code> — <i>Spotlight</i>	15
6.6. <code>test_4_6.xml</code> — Texturas	16
7. Sistema Solar	17
7.1. Iluminação solar	17
7.2. Estrutura hierárquica de cada planeta	18
7.3. Texturas dos corpos celestes	19
7.4. Estrutura XML (exemplo: Terra)	19
8. Otimizações	21
8.1. Cache partilhada de VBOs e texturas	21
8.2. Cache de estado GPU	21
8.3. <i>Frustum culling</i> hierárquico	21
9. Conclusão	22

2. Introdução

O presente relatório descreve o trabalho realizado na **4ª Fase do Trabalho Prático** da Unidade Curricular **Computação Gráfica**, pertencente ao 2º Semestre do 3º Ano da Licenciatura em Engenharia Informática, na Universidade do Minho, ano letivo 2024/2025.

Nesta fase foram abordados três tópicos fundamentais que transformam a cena de geometria simples numa renderização com aspeto realista:

- **Normais e coordenadas de textura:** o gerador passou a produzir, para cada vértice, um vetor normal e um par de coordenadas UV. O *engine* lê e envia estes dados para a GPU através de VBOs dedicados, ativando a indexação de vértices para eliminar duplicados.
- **Iluminação:** suporte a três tipos de fontes de luz definidas em XML — direcional, pontual e *spotlight* — aplicadas via o modelo de iluminação de Phong do OpenGL (*fixed-function pipeline*).
- **Materiais e texturas:** cada modelo pode ter associado um material com componentes difusa, ambiente, especular, emissiva e brilho (*shininess*), bem como uma textura carregada de ficheiro de imagem e aplicada via mapeamento UV.

Como cena de demonstração obrigatória, o sistema solar foi enriquecido com texturas realistas dos planetas e iluminação a partir do Sol.

3. Normais e Coordenadas de Textura

3.1. Formato do ficheiro `.3d`

O formato `.3d` foi estendido para incluir, em cada linha de vértice, o vetor normal e as coordenadas de textura:

Campo	Significado
<code><N></code>	Número total de vértices no ficheiro
<code>x y z</code>	Posição do vértice em coordenadas locais
<code>nx ny nz</code>	Vetor normal unitário (opcional — zero se ausente)
<code>u v</code>	Coordenadas de textura em $[0, 1]^2$ (opcional)

Tabela 1: Estrutura de cada linha do ficheiro `.3d` estendido

Os campos de normal e UV são opcionais — se ausentes, o *engine* usa zeros, mantendo compatibilidade com ficheiros gerados nas fases anteriores.

3.2. Geração de normais por primitiva

Cada primitiva calcula as normais analiticamente, garantindo suavidade e correção geométrica:

Primitiva	Normal por vértice
Plano	$\mathbf{n} = (0, 1, 0)$ — constante em toda a superfície
Esfera	$\mathbf{n} = \mathbf{p}/r$ — direção radial (posição normalizada)
Cone	$\mathbf{n} = (h \sin \alpha/l,; r/l,; h \cos \alpha/l)$ onde $l = \sqrt{r^2 + h^2}$
Cilindro	$\mathbf{n} = (\sin \alpha, 0, \cos \alpha)$ nas laterais; $\pm(0, 1, 0)$ nas bases
Caixa	$\mathbf{n}$ = normal da face: $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$ ou $(0, 0, \pm 1)$
Patch Bézier	Produto vetorial das derivadas parciais $\partial \mathcal{P}/\partial u \times \partial \mathcal{P}/\partial v$

Tabela 2: Cálculo de normais por primitiva geométrica

Para a esfera em coordenadas esféricas (α, β) (longitude, latitude):

$$\mathbf{n}(\alpha, \beta) = (\cos \beta \cos \alpha,; \sin \beta,; \cos \beta \sin \alpha)$$

Para o cone de raio r , altura h e comprimento de geratriz $l = \sqrt{r^2 + h^2}$, a normal na lateral ao ângulo azimutal α é:

$$\mathbf{n}(\alpha) = \left(\frac{h}{l} \sin \alpha,; \frac{r}{l},; \frac{h}{l} \cos \alpha \right)$$

Para os *patches* de Bézier bicúbicos, a posição na superfície e a normal são:

$$\mathcal{P}(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{i(u)} B_{j(v)} \mathbf{P}_{ij}, \quad \mathbf{n}(u, v) = \frac{\partial \mathcal{P}}{\partial u} \times \frac{\partial \mathcal{P}}{\partial v}$$

3.3. Coordenadas UV

As coordenadas de textura $(u, v) \in [0, 1]^2$ mapeiam a superfície de cada primitiva para o espaço de textura:

Primitiva	u	v
Esfera	$\alpha/(2\pi)$	$(\beta + \pi/2)/\pi$
Cone	ângulo azimutal $/(2\pi)$	y/h
Plano	x/L	z/L
Caixa	por face	cada face mapeada para $[0, 1]^2$

Tabela 3: Fórmulas de coordenadas de textura por primitiva

Na esfera, a projeção esférica equiretangular desenvolve a superfície como um mapa-mundo: u percorre a longitude e v a latitude, de polo a polo.

CPP

```

Vec3f norm = pos.Normalized();           // normal = posição normalizada
float u    = alpha / (2.0f * M_PI);      // longitude → [0, 1]
float v    = (beta + M_PI_2) / M_PI;     // latitude  → [0, 1]

```

3.4. Indexação de vértices

O *engine* elimina vértices duplicados usando uma tabela de hash cujas chaves incluem todos os atributos do vértice $(x, y, z, n_x, n_y, n_z, u, v)$. O resultado é um *index buffer* compacto que permite usar `glDrawElements` em vez de `glDrawArrays`, reduzindo o volume de dados enviados à GPU.

Entrada: sequência de vértices $V = \{(p, n, u, v)\}$ lida do ficheiro `.3d`

Saída: arrays P , N , UV (únicos) e buffer de índices I

$H \leftarrow \emptyset$ (tabela de hash: $(p, n, u, v) \rightarrow \text{índice}$)

Para cada vértice v em V :

Se $v \in H$:

$I.\text{adicionar}(H[v])$ (vértice já existe — reutiliza o índice)

Senão:

$\text{idx} \leftarrow |P|$

$P.\text{adicionar}(v.p)$; $N.\text{adicionar}(v.n)$; $UV.\text{adicionar}((v.u, v.v))$

$H[v] \leftarrow \text{idx}$; $I.\text{adicionar}(\text{idx})$

Renderizar com `glDrawElements(I)`

Listagem 1: Desduplicação de vértices com tabela de hash

Os dados geométricos são organizados em quatro VBOs independentes:

VBO	Tipo OpenGL	Conteúdo
<code>vbo_pos</code>	<code>GL_ARRAY_BUFFER</code>	Posições (x, y, z) — <code>GL_FLOAT × 3</code>
<code>vbo_norm</code>	<code>GL_ARRAY_BUFFER</code>	Normais (n_x, n_y, n_z) — <code>GL_FLOAT × 3</code>
<code>vbo_tex</code>	<code>GL_ARRAY_BUFFER</code>	Coordenadas UV (u, v) — <code>GL_FLOAT × 2</code>
<code>vbo_idx</code>	<code>GL_ELEMENT_ARRAY_BUFFER</code>	Índices de vértice — <code>GL_UNSIGNED_INT</code>

Tabela 4: Organização dos VBOs por tipo de dado geométrico

4. Iluminação

A iluminação é implementada através do modelo de Phong do *fixed-function pipeline* do OpenGL. O XML permite definir até 8 fontes de luz (limite do OpenGL), de três tipos diferentes.

4.1. Modelo de iluminação de Phong

A intensidade resultante num ponto da superfície é dada pela soma das contribuições de todas as N fontes de luz:

$$I = k_e + k_a \cdot I_a + \sum_{j=1}^N [k_d(l_j \cdot n) + k_s(r_j \cdot v)^s]$$

k_e	Componente emissiva — independente de qualquer fonte de luz
k_a, I_a	Componente ambiente e iluminação global do ambiente
k_d	Componente difusa — proporcional a $\cos \theta = \mathbf{l}_j \cdot \mathbf{n}$ (Lei de Lambert)
k_s	Componente especular — reflexo de Phong com expoente s
$\mathbf{r}_j$	Vetor de reflexão da luz j em relação à normal $\mathbf{n}$
$\mathbf{v}$	Vetor de visão (direção para a câmara)

Tabela 5: Termos do modelo de iluminação de Phong

4.2. Definição em XML

```

<lights>
  <light type="directional" dirX="1" dirY="0.7" dirZ="0.5" />
  <light type="point"      posX="0" posY="10" posZ="0" />
  <light type="spot"       posX="0" posY="10" posZ="0"
                           dirX="0" dirY="-1" dirZ="0"
                           cutoff="30" />
</lights>

```

O *parser* aceita tanto os atributos em *camelCase* (`posX`, `dirX`) como em minúsculas (`posx`, `dirx`) para compatibilidade com diferentes versões dos ficheiros de teste.

4.3. Tipos de luz

4.3.1. Luz direcional

Simula uma fonte infinitamente distante (ex.: Sol numa cena exterior). A direção é constante em toda a cena e a intensidade não sofre atenuação com a distância. O OpenGL distingue os tipos de luz pelo quarto componente w da posição homogênea enviada a `GL_POSITION`:

$$\mathbf{p}_h = \begin{pmatrix} d_x \\ d_y \\ d_z \\ w = 0 \end{pmatrix} \Rightarrow \text{fonte no infinito}$$

```

float dir4[4] = {dl.dir.x, dl.dir.y, dl.dir.z, 0.0f};
glLightfv(GL_LIGHT0 + i, GL_POSITION, dir4);

```

4.3.2. Luz pontual

Emite luz em todas as direções a partir de uma posição finita no espaço. Com $w = 1$, o OpenGL trata o vetor como posição:

$$\mathbf{p}_h = \begin{pmatrix} p_x \\ p_y \\ p_z \\ w = 1 \end{pmatrix} \Rightarrow \text{fonte em posição finita}$$

```
float pos4[4] = {pl.pos.x, pl.pos.y, pl.pos.z, 1.0f};
glLightfv(GL_LIGHT0 + i, GL_POSITION, pos4);
```

A intensidade decresce com a distância d segundo a atenuação quadrática:

$$I(d) = \frac{I_0}{k_c + k_l d + k_q d^2}$$

onde k_c , k_l e k_q são os coeficientes constante, linear e quadrático, respetivamente.

4.3.3. Spotlight

Fonte pontual com cone de iluminação limitado pelo ângulo de *cutoff* φ_c . Um ponto é iluminado apenas se o ângulo θ entre o vetor de incidência $\mathbf{l}$ e a direção do *spot* $\mathbf{d}$ satisfaz:

$$\theta = \arccos(\mathbf{l} \cdot \mathbf{d}) \leq \varphi_c$$

Dentro do cone, a intensidade é ainda modulada pelo expoente de *spotlight*:

$$I_{\text{spot}} = (\mathbf{l} \cdot \mathbf{d})^{\text{expoente}} \times I_{\text{pontual}}$$

Tipo	Parâmetros XML	w	Atenuação
Direcional	dirX dirY dirZ	0	Nenhuma
Pontual	posX posY posZ	1	Quadrática com a distância
Spotlight	posX posY posZ dirX dirY dirZ cutoff	1	Quadrática + cone angular

Tabela 6: Comparação dos três tipos de luz suportados

4.4. Propriedades de Reflexão

Cada modelo pode ter um bloco `<color>` no XML que define os coeficientes de reflexão usados pelo OpenGL no cálculo da iluminação:

```
<color>
  <diffuse R="0"    G="87"   B="184" />
  <ambient R="0"    G="8"    B="18"  />
  <specular R="255"  G="255"  B="255" />
  <emissive R="0"    G="0"    B="0"   />
  <shininess value="128" />
</color>
```


Os valores RGB são definidos em inteiros $[0, 255]$ e convertidos para componentes *float* $[0.0, 1.0]$ antes de serem enviados ao OpenGL:

$$c_i = \frac{R_i}{255}, \quad i \in \{R, G, B\}, \quad c_\alpha = 1.0$$

As cinco componentes têm os seguintes papéis no modelo de Phong:

Componente	Papel no modelo de Phong
Difusa k_d	Cor base — proporcional ao cosseno do ângulo de incidência (Lei de Lambert)
Ambiente k_a	Iluminação mínima independente da direção da luz
Especular k_s	Reflexo brilhante dependente da posição do observador
Emissiva k_e	Luz própria do objeto — soma-se independentemente de qualquer fonte
Brilho s	Expoente de Phong $s \in [0, 128]$ — controla a largura do reflexo especular

Tabela 7: Componentes do modelo de material de Phong

Para evitar chamadas redundantes ao driver a cada *frame*, o *engine* mantém um cache do último material enviado à GPU e só o reenvia quando este muda:

Estado: $M_{\text{cache}} \leftarrow$ último material enviado; válido: booleano

Para cada modelo a renderizar:

Se M_{cache} inválido **ou** $M_{\text{atual}} \neq M_{\text{cache}}$:

 Enviar difusa, ambiente, especular, emissiva e brilho para a GPU

$M_{\text{cache}} \leftarrow M_{\text{atual}}$

Senão:

 (nenhuma chamada ao driver — estado GPU já é o correto)

Figura 1: Cache de estado de material — minimiza chamadas ao driver

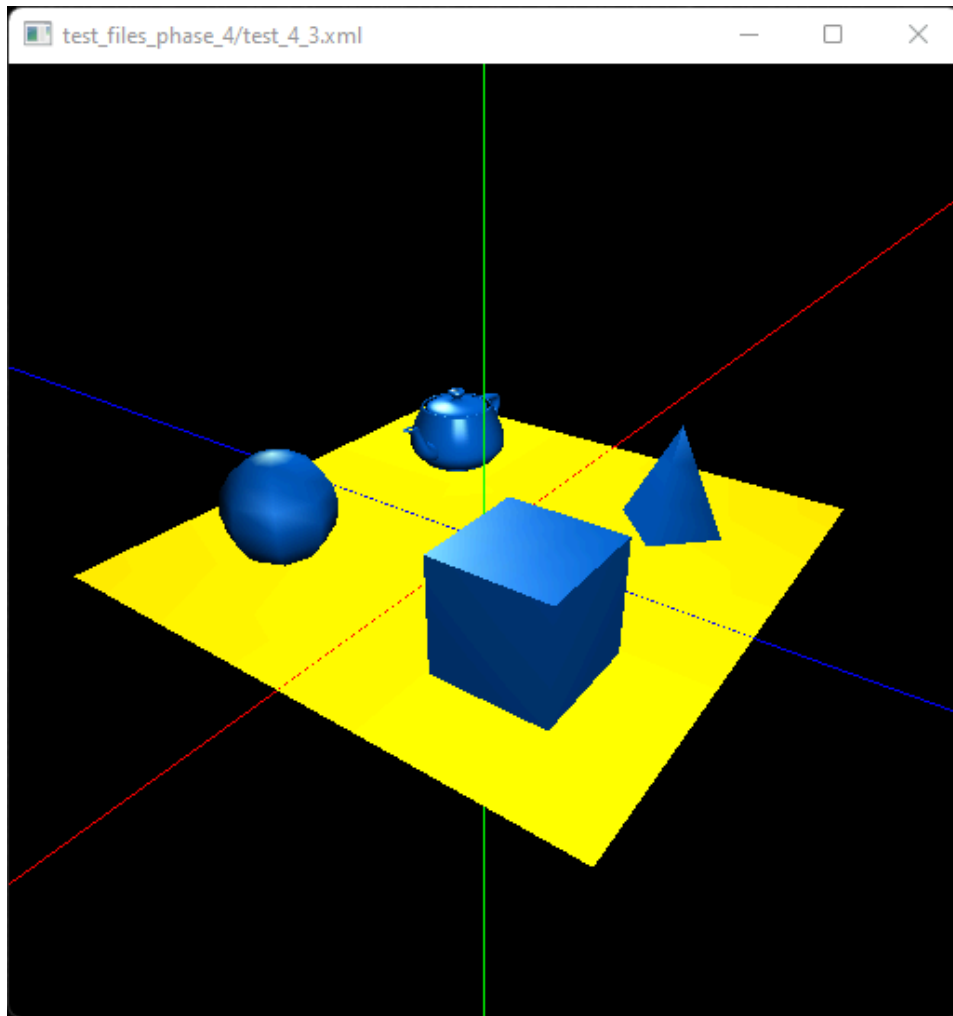


Figura 2: Materiais com componente especular — shininess = 128

5. Texturas

5.1. Carregamento

As texturas são carregadas a partir de ficheiros de imagem (JPEG, PNG, etc.) usando a biblioteca *stb_image*. O processo segue os seguintes passos:

1. **Inversão vertical:** a origem do espaço UV do OpenGL está no canto inferior-esquerdo, mas a maioria dos ficheiros de imagem usa o canto superior-esquerdo. A coordenada v é invertida na leitura para corrigir esta discrepância.
2. **Descodificação:** a imagem é lida com resolução $w \times h$ e $c \in \{3, 4\}$ canais (RGB ou RGBA).
3. **Geração do *texture object*:** é criado um identificador de textura na GPU.
4. **Configuração dos filtros de amostragem:**
 - *Minificação:* trilinear com *mipmaps* (`GL_LINEAR_MIPMAP_LINEAR`) — elimina *aliasing* em objetos distantes.
 - *Magnificação:* bilinear (`GL_LINEAR`) — suaviza a ampliação.

5. **Modo de repetição:** `GL_REPEAT` em ambos os eixos S e T .
6. **Transferência para a GPU:** os pixels são enviados para memória de vídeo no formato RGB ou RGBA conforme c .
7. **Geração de *mipmaps*:** a cadeia completa de *mipmaps* é gerada automaticamente a partir do nível base.

5.2. Cache de texturas

As texturas são partilhadas entre todos os modelos que referenciam o mesmo ficheiro — o caminho do ficheiro serve como chave de cache. A transferência para a GPU ocorre apenas na primeira utilização:

Estado: T_{cache} : mapa (caminho $\rightarrow$ identificador GPU)

Ao carregar textura com caminho p :

Se $p \in T_{\text{cache}}$:

Reutilizar $T_{\text{cache}[p]}$ (sem nova transferência para a GPU)

Senão:

Executar passos 1–7 do algoritmo de carregamento

$T_{\text{cache}[p]} \leftarrow$ identificador gerado

Figura 3: Cache de texturas — partilha entre instâncias

5.3. Definição em XML

A textura é associada a cada modelo individualmente; o material serve de *fallback* caso o ficheiro de textura não seja encontrado:

```
<model file="assets/models/sphere.3d">
  <texture file="2k_earth_daymap.jpg" />
  <color>
    <diffuse R="200" G="200" B="200" />
    <ambient R="50" G="50" B="50" />
  </color>
</model>
```

O *engine* procura o ficheiro em múltiplos caminhos predefinidos (`assets/textures/`, caminhos relativos ao binário, etc.), garantindo portabilidade independente do diretório de execução.

5.4. Aplicação na renderização

Durante a renderização, o *binding* de textura é controlado por um cache de estado à semelhança do cache de material: a chamada `glBindTexture` é emitida apenas quando a textura muda entre modelos consecutivos.

Os estados OpenGL necessários — `GL_TEXTURE_2D`, `GL_VERTEX_ARRAY`, `GL_NORMAL_ARRAY` e `GL_TEXTURE_COORD_ARRAY` — são ativados uma única vez durante a inicialização e permanecem ativos durante toda a sessão, evitando chamadas redundantes a `glEnable`/`glEnableClientState` por *frame*.

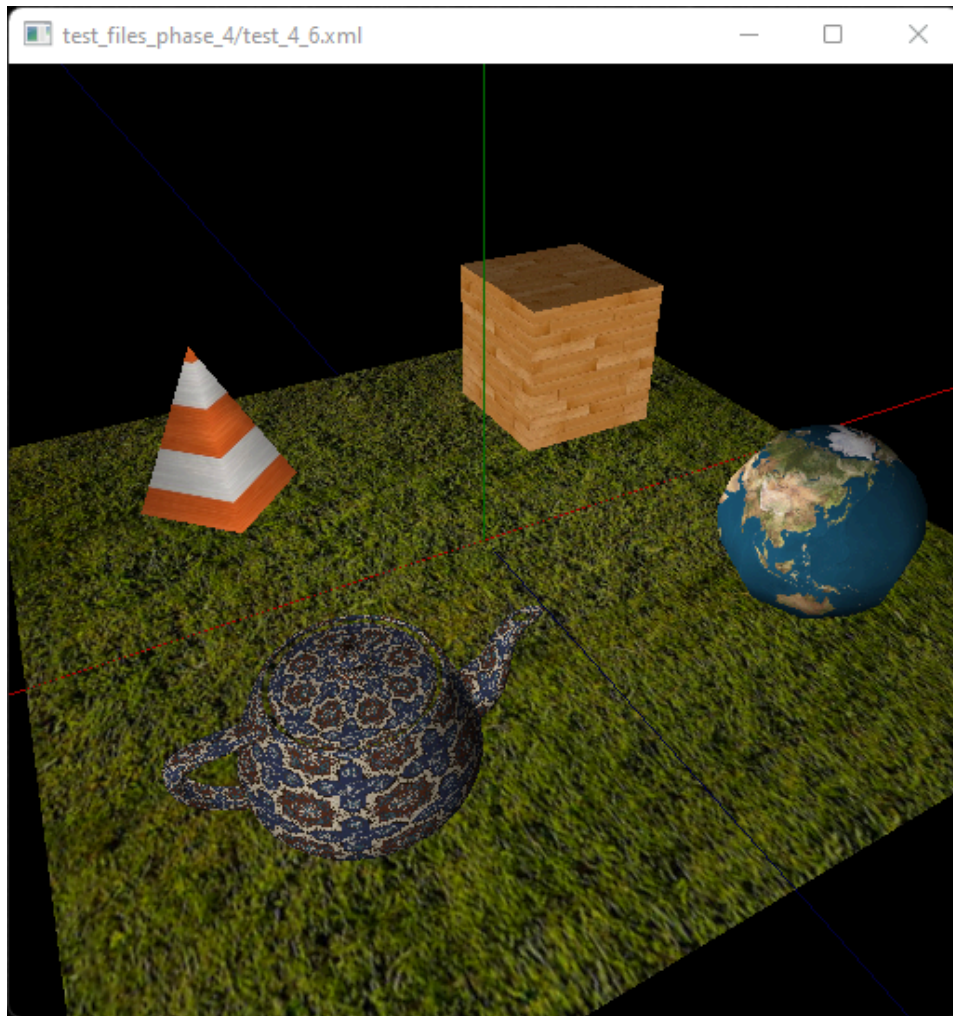


Figura 4: Cena com texturas aplicadas via mapeamento UV

6. Testes

Os seis testes desta fase validam progressivamente a iluminação, os materiais e as texturas.

6.1. test_4_1.xml — Luz direcional, cor difusa

Cena mínima com um cubo iluminado por uma única luz direcional com direção $(1, 0.7, 0.5)$ e material de difusa amarela, sem componentes especular ou emissiva. A variação de intensidade Lambertiana entre as faces torna visível a orientação de cada face em relação à fonte de luz.

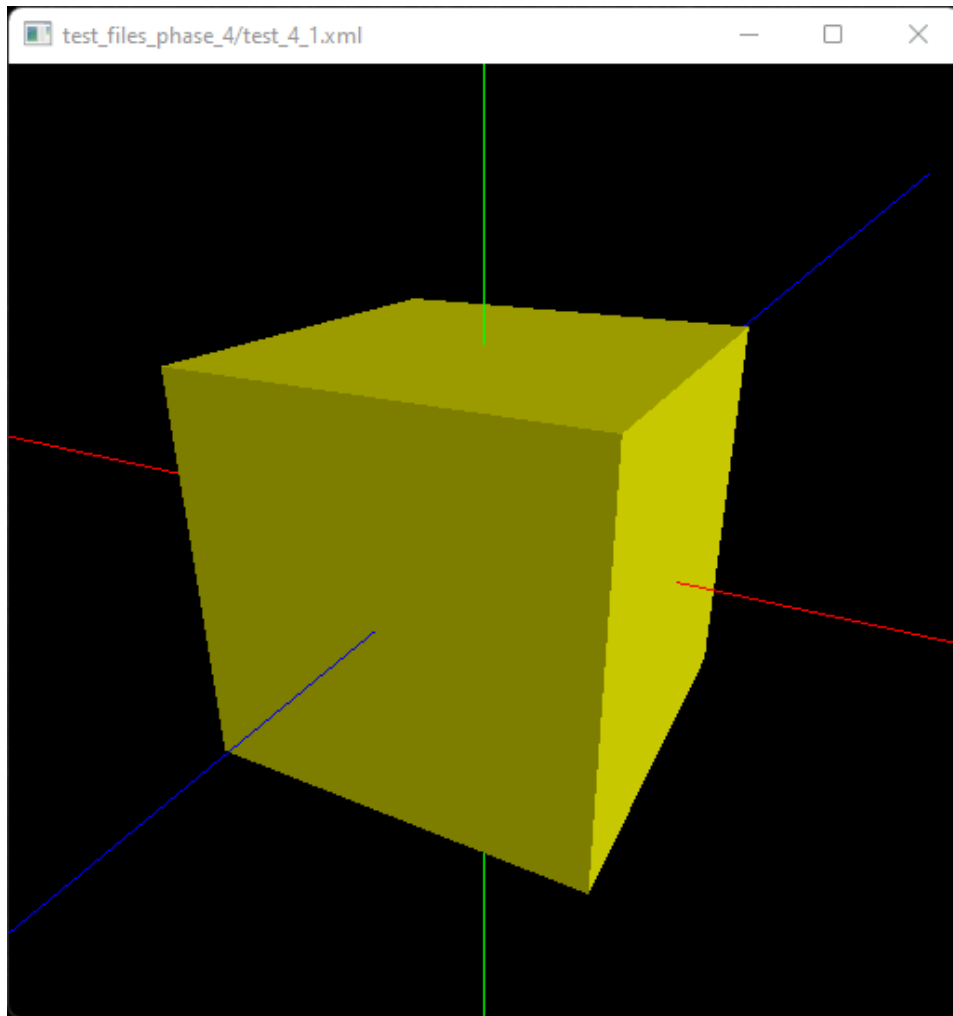


Figura 5: *test_4_1.xml* — cubo com luz direcional e material difuso amarelo

6.2. *test_4_2.xml* — Componente emissiva

Idêntico ao anterior mas com componente emissiva vermelha intensa. A emissiva soma-se ao amarelo difuso resultando num tom laranja mesmo nas faces menos iluminadas — evidencia que a componente k_e é independente de qualquer fonte de luz.

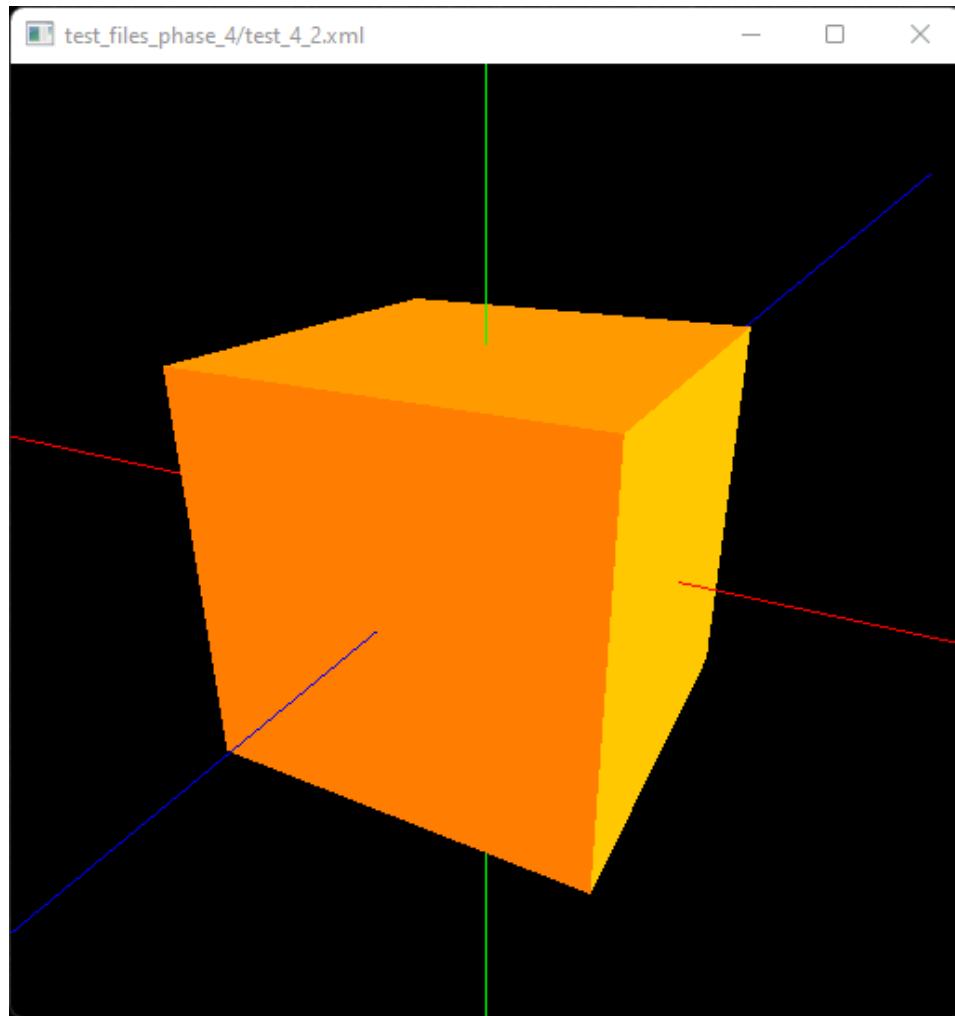


Figura 6: *test_4_2.xml* — efeito da componente emissiva na cor final

6.3. *test_4_3.xml* — Dois *point lights*, especular

Plano com cone, esfera, bule e caixa com material azul e *shininess* = 128, iluminados por dois pontos de luz em posições simétricas. O expoente de Phong elevado produz reflexos especulares concentrados e brilhantes nos objetos curvos.

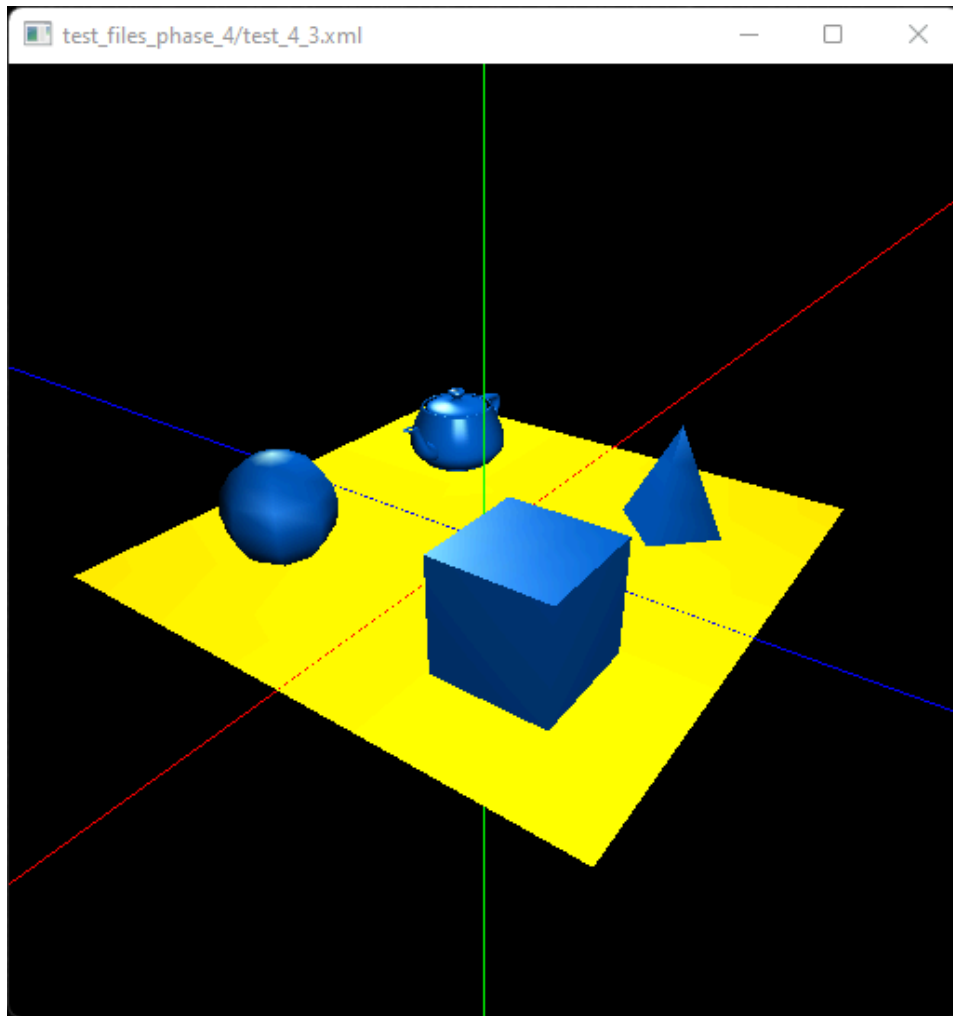


Figura 7: *test_4_3.xml* — reflexos especulares com dois point lights

6.4. *test_4_4.xml* — *Point light* central

Mesma cena mas com uma única luz pontual muito próxima da origem $(0, 0.2, 0)$. A atenuação quadrática $I(d) \propto \frac{1}{d^2}$ produz contraste dramático — objetos próximos da fonte ficam muito mais brilhantes que os afastados.

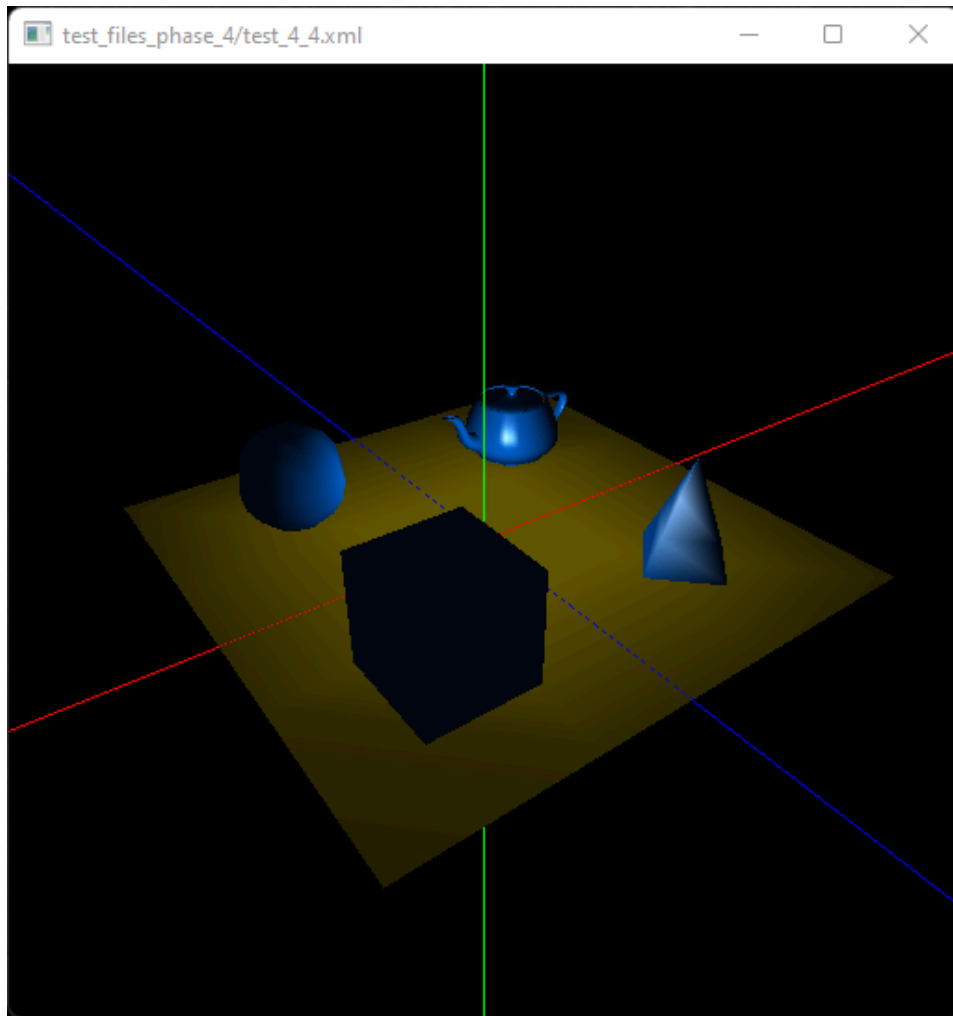


Figura 8: *test_4_4.xml* — atenuação quadrática com point light central

6.5. *test_4_5.xml* — *Spotlight*

Spotlight com ângulo de *cutoff* $\varphi_c = 10^\circ$ posicionado acima da cena e apontando para a origem. Apenas os objetos dentro do cone ($\theta \leq \varphi_c$) recebem iluminação — as bordas da cena ficam completamente escuras.

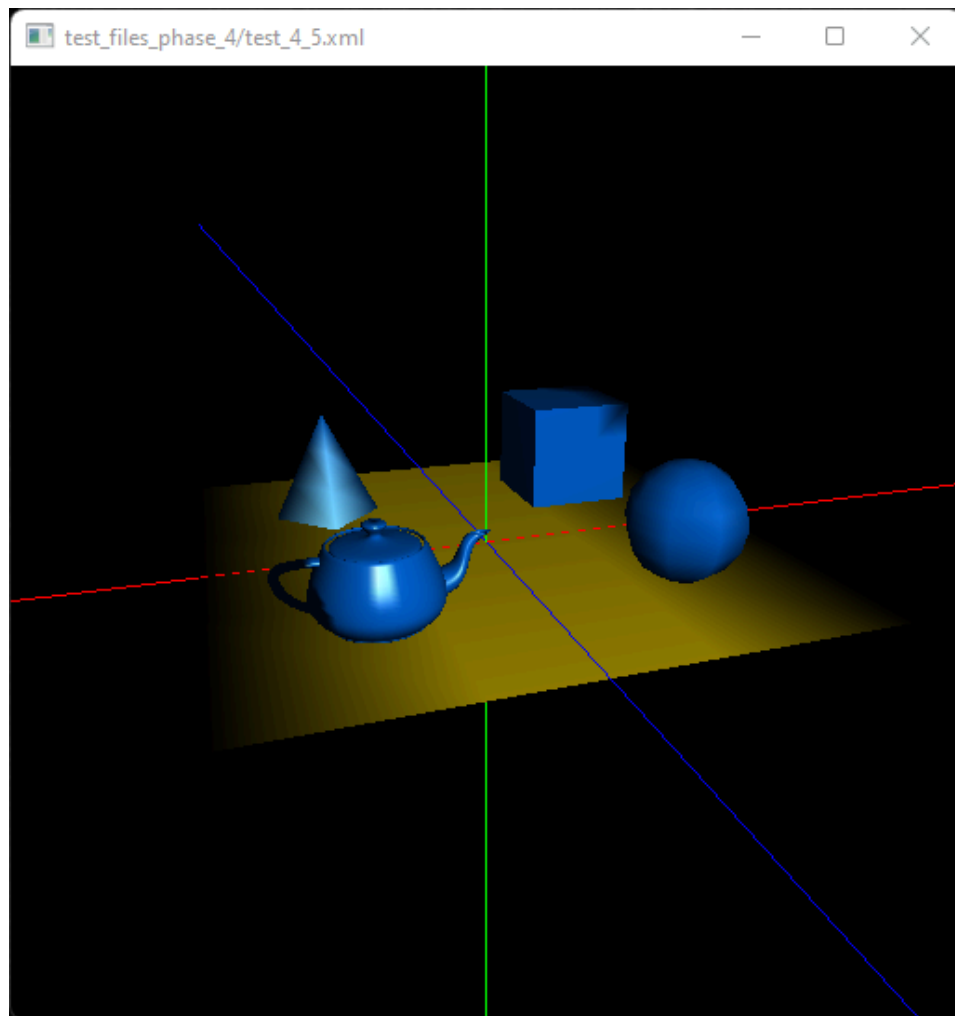


Figura 9: *test_4_5.xml* — spotlight com cutoff de 10°

6.6. *test_4_6.xml* — Texturas

Mesma disposição de objetos mas com texturas em todos os modelos: relva no plano, cone de trânsito, globo terrestre, bule cerâmico e caixote de madeira. Dois *point lights* garantem iluminação suficiente para as texturas serem visíveis com os seus detalhes e cores.

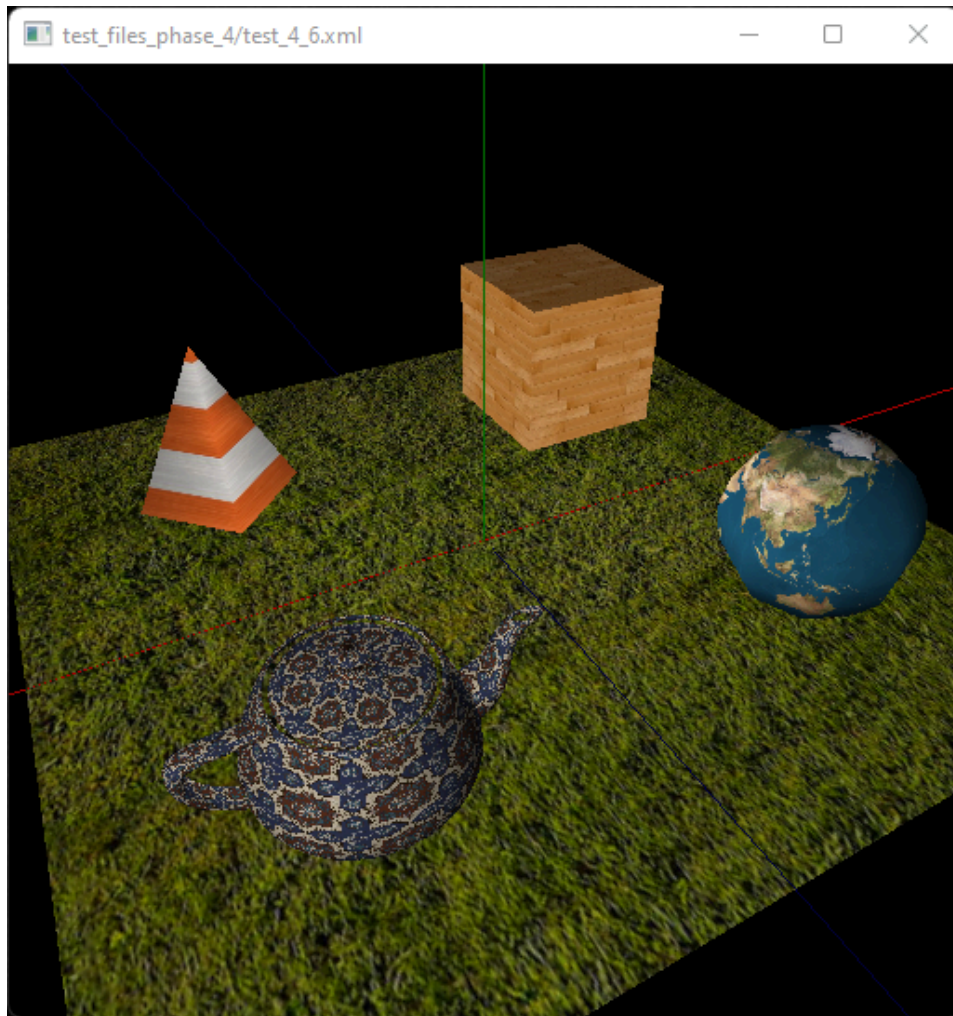


Figura 10: test_4_6.xml — cena com texturas e iluminação

7. Sistema Solar

O sistema solar foi enriquecido com texturas realistas e iluminação a partir do Sol, mantendo todas as animações da Fase 3.

7.1. Iluminação solar

O sistema solar usa quatro fontes de luz complementares, definidas no XML:

Tipo	Posição / Direção	Papel
Pontual	origem (0, 0, 0) — Sol	Luz principal, amarela quente; simula a radiação solar com atenuação quadrática
Direcional	(1,; 0.3,; 0.5)	Luz de preenchimento fria para iluminar ligeiramente o lado escuro dos planetas
<i>Spotlight</i>	perto de Saturno, cutoff = 20°	Feixe azul-branco direcionado a Saturno e aos seus anéis
Pontual	(−200,; 50,; −200)	<i>Rim light</i> de fundo para evitar silhuetas completamente negras

Tabela 8: Fontes de luz do sistema solar

A luz pontual do Sol é a dominante: a sua atenuação quadrática com a distância produz um gradiente de brilho realista do interior para o exterior do sistema. As restantes três luzes são auxiliares e de baixa intensidade, garantindo que nenhuma face fica completamente negra e destacando visualmente Saturno.

7.2. Estrutura hierárquica de cada planeta

Cada planeta é representado por dois grupos aninhados, separando as transformações de órbita e de auto-rotação:

Grupo	Transformações aplicadas
Exterior	Rotação orbital em torno do Sol + translação para a distância orbital
Interior	Auto-rotação em torno do próprio eixo, com período independente da órbita

Tabela 9: Estrutura de dois grupos aninhados por planeta

Esta separação permite animar órbita e rotação independentemente, sem acumulação de erros de ponto flutuante.

7.3. Texturas dos corpos celestes

Corpo celeste	Ficheiro de textura
Skybox (Via Láctea)	<code>2k_stars_milky_way.jpg</code> — esfera invertida emissiva a envolver toda a cena
Sol	<code>2k_sun.jpg</code> — material emissivo amarelo
Mercúrio	<code>2k_mercury.jpg</code>
Vénus	<code>2k_venus_surface.jpg</code>
Terra	<code>2k_earth_daymap.jpg</code>
Lua (Terra)	<code>2k_moon.jpg</code>
Marte	<code>2k_mars.jpg</code>
Fobos / Deimos	<code>2k_moon.jpg</code>
Júpiter	<code>2k_jupiter.jpg</code>
Luas de Júpiter	<code>2k_moon.jpg</code> (2 luas)
Saturno	<code>2k_saturn.jpg</code>
Anel de Saturno	<code>saturnringcolor.jpg</code> — torus com textura de anel
Titan	<code>2k_moon.jpg</code>
Urano	sem textura — material azul-ciano com componente emissiva
Anel de Urano	<code>uranusringcolour.jpg</code> — inclinado 98° sobre o eixo Z
Neptuno	<code>neptunemap.jpg</code>
Plutão	<code>plutomap1k.jpg</code>
Cometa	<code>2k_moon.jpg</code> — material com componente emissiva azul-branca

Tabela 10: Texturas associadas a cada corpo celeste do sistema solar

7.4. Estrutura XML (exemplo: Terra)

XML

```

<group>
  <transform>
    <rotate time="10" x="0" y="1" z="0" /> <!-- órbita anual -->
  </transform>
  <transform>
    <translate x="19" y="0" z="0" /> <!-- distância ao Sol -->
  </transform>
  <group>
    <transform>
      <rotate time="2" x="0" y="1" z="0" /> <!-- rotação diária -->
    </transform>
    <models>
      <model file="assets/models/sphere.3d">
        <texture file="2k_earth_daymap.jpg" />
        <color>

```

```

    <diffuse R="120" G="160" B="220" />
    <ambient R="30" G="40" B="55" />
    <specular R="150" G="160" B="200" />
    <shininess value="80" />
  </color>
</model>
</models>
</group>
</group>

```

Os dois blocos `<transform>` são intencionais: o primeiro aplica a rotação orbital (antes de qualquer translação), o segundo posiciona o planeta à distância correta. Esta separação resulta em órbitas circulares centradas na origem sem acumulação de erros de composição de matrizes.

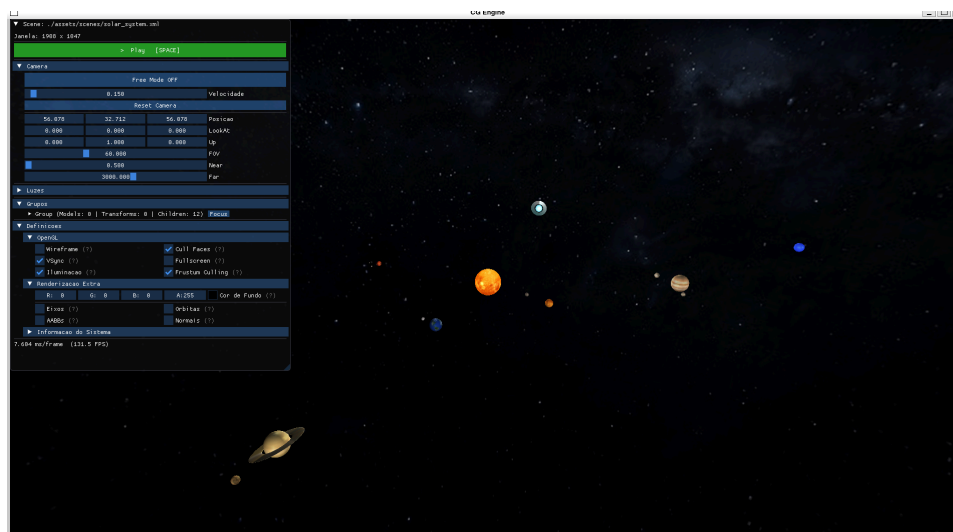


Figura 11: Sistema solar — mapeamento de texturas e modelo de iluminação global

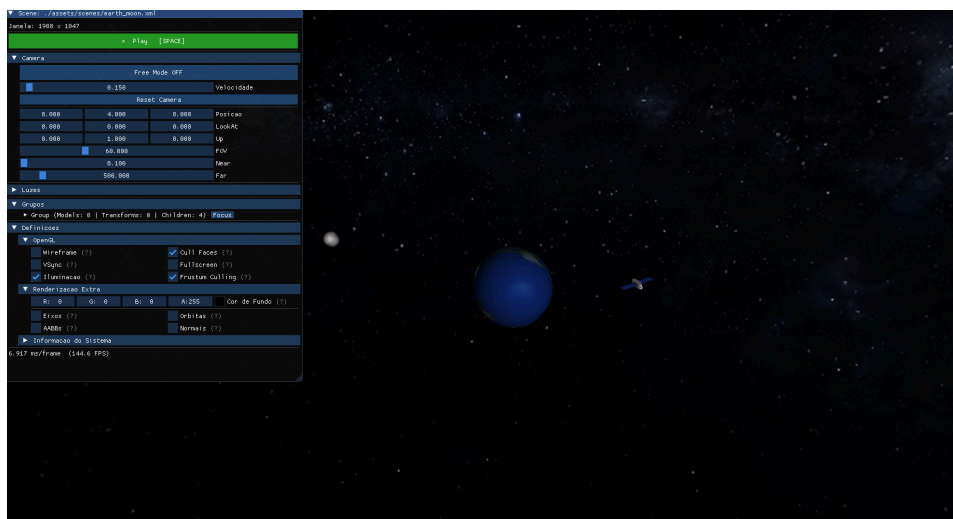


Figura 12: Terra — textura de superfície com sombreamento difuso e especular

8. Otimizações

Além das funcionalidades obrigatórias, foram implementadas várias otimizações que melhoram significativamente o desempenho em cenas complexas.

8.1. Cache partilhada de VBOs e texturas

Modelos e texturas referenciados múltiplas vezes na cena (ex.: `sphere.3d` usado por todos os planetas) são carregados para a GPU uma única vez. As instâncias subsequentes reutilizam os mesmos identificadores de VBO e textura, sem nova transferência de dados.

Recurso	Primeiro acesso (cache miss)	Acessos seguintes (cache hit)
Geometria	Ler <code>.3d</code> , criar e preencher 4 VBOs na GPU	Copiar referências dos VBOs já existentes
Textura	Decodificar imagem, transferir para GPU	Reutilizar identificador GPU

Tabela 11: Comportamento da cache em primeiro e segundo acesso ao mesmo recurso

No sistema solar, os 8 planetas partilham o mesmo VBO de esfera — a geometria é enviada para a GPU apenas uma vez, independentemente do número de instâncias.

8.2. Cache de estado GPU

Para cada tipo de recurso ligável (VBOs, textura, material), o *engine* mantém o identificador do último valor enviado ao driver. Uma chamada de *binding* ou atualização só é emitida quando o valor muda entre modelos consecutivos:

Cache	Evita chamada redundante ao driver
VBO de posições	<code>glBindBuffer</code> + <code>glVertexPointer</code> quando mesmo modelo é consecutivo
VBO de normais	<code>glBindBuffer</code> + <code>glNormalPointer</code> idem
VBO de coordenadas	<code>glBindBuffer</code> + <code>glTexCoordPointer</code> idem
Buffer de índices	<code>glBindBuffer</code> do <code>GL_ELEMENT_ARRAY_BUFFER</code> idem
Textura	<code>glBindTexture</code> entre modelos com a mesma textura
Material	<code>glMaterialfv</code> entre modelos com o mesmo material

Tabela 12: Caches de estado GPU mantidas pelo engine por frame

8.3. Frustum culling hierárquico

O *engine* calcula uma AABB (*Axis-Aligned Bounding Box*) em coordenadas mundo para cada grupo da hierarquia de cena e testa-a contra os seis planos do *frustum* de visão. Grupos completamente fora de algum plano são descartados inteiramente, incluindo todos os seus descendentes, sem emitir qualquer *draw call*.

A técnica de *plane masking* (máscara de planos) reduz adicionalmente o número de testes: quando a AABB de um grupo está completamente dentro de um plano, esse plano é removido da máscara dos descendentes e não é retestado:

Entrada: grupo G ; máscara herdada M (6 bits — um por plano do *frustum*)

Invariante: bit $M[i] = 1$ indica que o plano i ainda não foi satisfeito

$M_{\text{filho}} \leftarrow M$

Para cada plano p_i com $M[i] = 1$:

 Testar $\text{AABB}(G)$ contra p_i :

Totalmente fora: retornar (G e toda a descendência descartados)

Totalmente dentro: $M_{\text{filho}}[i] \leftarrow 0$ (*plano satisfeito — não retestar*)

Intersecta: $M_{\text{filho}}[i]$ mantém-se 1

Renderizar modelos de G

Para cada filho F de G :

$\text{Render}(F, M_{\text{filho}})$ (*máscara reduzida — menos testes por nível*)

Figura 13: Frustum culling hierárquico com plane masking

9. Conclusão

Nesta fase foram implementadas com sucesso todas as funcionalidades propostas:

- **Normais e UV:** o gerador produz normais analíticas e coordenadas de textura para todas as primitivas. O *engine* lê estes dados, elimina vértices duplicados via tabela de hash e envia para a GPU através de VBOs indexados.
- **Iluminação:** os três tipos de luz (direcional, pontual e *spotlight*) são definidos em XML e aplicados via o modelo de Phong, com suporte a até 8 fontes simultâneas.
- **Materiais:** as cinco componentes de Phong (difusa, ambiente, especular, emissiva, brilho) são configuráveis por modelo em XML e aplicadas com cache de estado para minimizar chamadas ao driver.
- **Texturas:** carregamento com *stb_image* e *mipmaps* automáticos, partilha de texturas entre instâncias e aplicação via mapeamento UV gerado analiticamente para cada primitiva.

As otimizações implementadas — cache partilhada de VBOs e texturas, cache de estado GPU e *frustum culling* hierárquico com *plane masking* — permitem manter taxas de *frame* elevadas mesmo com o sistema solar completo: 19 corpos celestes com texturas individuais, anéis de Saturno e Urano, skybox da Via Láctea, luas animadas e 4 fontes de luz simultâneas.